



DESAFIO 2

INTELIGÊNCIA

COMPUTACIONAL

JONATHAN MOREIRA BARREIROS - 12421BCC034

KAIKE DE MORAIS CARVALHO - 12421BCC051



CONTEXTO:

O sinal que passa por cada camada da rede neural é composto pelo produto interno entre os pesos e os dados de entrada, somados ao viés e transformados por uma função de ativação.

O acúmulo da variância desse sinal ao longo da sua propagação pelas camadas pode fazer com que o sinal suma ou exploda.

Variância indo a zero = neurônios convergem pro mesmo valor, fazendo com que as camadas empilhadas percam sua representatividade sobre os dados. Rede “esquece” da informação de entrada nas camadas profundas.

Variância explodindo = neurônios saturados, aprendizado é incapacitado.

A inicialização adequada dos pesos é o que decide se a informação (e o seu erro a ser corrigido) vai conseguir passar por todas as camadas.

Além disso, a função de ativação é a responsável por aplicar um efeito de escala sobre a variância propagada: o RELU zera os valores negativos, reduzindo a variância pela metade; o tanh se comporta quase de forma linear para valores próximos de zero, sendo vantajoso pra retropropagação dos gradientes.



OBJETIVO:

Implementar uma função de ativação e uma de inicialização dos pesos de forma adequada para preservar a variância do sinal propagado durante o treinamento do modelo.

in-place: os valores para os pesos w e viés b são inseridos diretamente no tensor

elemento a elemento, sem lote: função de ativação deve aplicar sobre cada posição do vetor de forma independente, sem aplicar sobre o batch (que é o que acontece no BatchNorm/Normalização em Lote).

diferenciável: especifica o uso de funções nativas do PyTorch para garantir diferenciabilidade, o que torna o treinamento possível por conta do backpropagation (autograd).

camada, n_camadas: diz qual camada os pesos estão sendo inicializados e qual a quantidade de camadas da rede, permitindo variar a escala e “intensidade” da função de ativação conforme a profundidade.

raio-x: var e ||grad||: exibe a variância da ativação e a norma do gradiente, camada por camada.





NOSSA ABORDAGEM:

Inicializar o tensor de pesos de forma ortogonal ao invés de uniforme (utilizada no baseline e exemplo).

Ao invés de gerar pesos aleatoriamente, a matriz ortogonal garante que as linhas/colunas sejam todas perpendiculares entre si e de norma 1.

Dessa forma, **uma transformação linear ortogonal preserva a norma de qualquer vetor.**

$$||\mathbf{Q}^*\mathbf{h}|| = ||\mathbf{h}|| \text{ para qualquer } \mathbf{h}$$

Com a inicialização gaussiana, a variância média provoca os desvios que se acumulam e degradam a propagação do sinal nas camadas.

Já com a inicialização ortogonal, cada componente $\mathbf{z} = \mathbf{Q}^*\mathbf{h}$ possui variância exatamente igual à variância de $\mathbf{h}$.

Isso significa que o sinal atravessa as camadas de forma exata, sem sofrer ação média. Assim, reduzindo o desvio.





NOSSA ABORDAGEM:

Porém, só a inicialização ortogonal dos pesos não impede que a função de ativação comprima os valores, o que ainda assim causaria o sumiço do sinal propagado pelas camadas.

Por conta disso, utilizamos um valor de ganho associado à função de ativação escolhida (**tanh**) para compensar essa “parcela perdida” dos valores que passam pela função.

$$||(\text{gain} * Q) * \mathbf{h}|| = \text{gain} * ||\mathbf{h}||$$

Por conta disso, utilizamos um valor de ganho associado à função de ativação escolhida (**tanh**) para compensar essa “parcela perdida” dos valores que passam pela função.

Além disso, verificamos se a camada sobre qual a inicialização dos pesos está sendo aplicada é a última camada. Caso seja a última camada, responsável por entregar os valores para a função de perda sem passar pela função de ativação, diminuimos o valor do ganho para reduzir a variância dos valores de saída.

Dessa forma, a função de peso produz uma distribuição quase uniforme entre classes.



NOSSA IMPLEMENTAÇÃO:

```
import math
import torch

def ativacao(x: torch.Tensor) -> torch.Tensor:
    return torch.nn.functional.tanh(x)          # elemento a elemento, sem parâmetros

@torch.no_grad()
def inicializar(W: torch.Tensor, b: torch.Tensor,
               fan_in: int, fan_out: int, camada: int, n_camadas: int) -> None:
    if camada == n_camadas:
        gain = 0.1 # camada de logits: um pouco menor ajuda o SGD
    else:
        gain = torch.nn.init.calculate_gain("tanh")
    torch.nn.init.orthogonal(W, gain=gain)
    b.zero_()
```

RAIO-X

Considerando que utilizamos o processamento --rapido

1

Baseline do professor

```
PS C:\Users\Kaike\Desktop\VsCode\IC> python harness_desafio2.py exemplo_submissao_d2.py --rapido
Desafio 2 – Ativação e inicialização em redes profundas (torch 2.14.0+cpu, cpu)
MLP largura 256, L ∈ (4, 16, 48), SGD lr=0.05 momento=0.9, 1 época(s), 1 semente(s), modo rápido

Submissão: exemplo_submissao_d2.py
mnist_L4      acc=0.875 base=0.876 ref=0.890 s_t=0.00
               var(pré-ativ) 1ª/meio/última: 2.4 3.4 0.67 | ||grad W||: 2.2 1.3 2.4
mnist_L16     acc=0.667 base=0.126 ref=0.442 s_t=1.25
               var(pré-ativ) 1ª/meio/última: 2.4 11 6.1 | ||grad W||: 5.9 6.5 17
mnist_L48     acc=0.100 base=0.126 ref=0.126 s_t=0.00
fashion_L4    acc=0.815 base=0.765 ref=0.808 s_t=1.16
               var(pré-ativ) 1ª/meio/última: 2.3 3 0.73 | ||grad W||: 2.7 1.7 3.3
fashion_L16   acc=0.100 base=0.095 ref=0.361 s_t=0.02
               var(pré-ativ) 1ª/meio/última: 2.2 11 5 | ||grad W||: 8 8.5 21
fashion_L48   acc=0.100 base=0.095 ref=0.100 s_t=1.00
               var(pré-ativ) 1ª/meio/última: 2.3 2.6e+02 3.2e+03 | ||grad W||: 1.4e+02 2.6e+02 9.8e+02
cifar10_L4    acc=0.290 base=0.340 ref=0.229 s_t=0.00
               var(pré-ativ) 1ª/meio/última: 2.3 2.8 0.92 | ||grad W||: 4 1.3 2.7
cifar10_L16   acc=0.145 base=0.100 ref=0.150 s_t=0.90
               var(pré-ativ) 1ª/meio/última: 2.1 2.7 2.2 | ||grad W||: 10 3.7 9.8
cifar10_L48   acc=0.100 base=0.100 ref=0.090 s_t=0.00
               var(pré-ativ) 1ª/meio/última: 2.3 95 3.4e+02 | ||grad W||: 2.1e+02 1.3e+02 3.1e+02
-----
ESCORE S = 33.7 (média=0.48, mínimo=0.00)
```

2

Baseline nosso

```
PS C:\Users\Kaike\Desktop\VsCode\IC> python harness_desafio2.py desafio2_jonathan_kaike.py --rapido
Desafio 2 – Ativação e inicialização em redes profundas (torch 2.14.0+cpu, cpu)
MLP largura 256, L ∈ (4, 16, 48), SGD lr=0.05 momento=0.9, 1 época(s), 1 semente(s), modo rápido

Submissão: desafio2_jonathan_kaike.py
C:\Users\Kaike\Desktop\VsCode\IC\desafio2_jonathan_kaike.py:23: FutureWarning: `nn.init.orthogonal` is now deprecated in favor of `nn.init.orthogonal_`.
  torch.nn.init.orthogonal(W, gain=gain)
C:\Users\Kaike\Desktop\VsCode\IC\desafio2_jonathan_kaike.py:23: FutureWarning: `nn.init.orthogonal` is now deprecated in favor of `nn.init.orthogonal_`.
  torch.nn.init.orthogonal(W, gain=gain)
mnist_L4      acc=0.891 base=0.876 ref=0.890 s_t=1.07
               var(pré-ativ) 1ª/meio/última: 2.9 1.2 0.0045 | ||grad W||: 0.21 0.11 1.7
mnist_L16     acc=0.828 base=0.126 ref=0.442 s_t=1.25
               var(pré-ativ) 1ª/meio/última: 2.8 1.2 0.0041 | ||grad W||: 0.45 0.12 1.3
mnist_L48     acc=0.200 base=0.126 ref=0.126 s_t=1.25
               var(pré-ativ) 1ª/meio/última: 2.8 1.2 0.0039 | ||grad W||: 8.9 0.47 0.91
fashion_L4    acc=0.812 base=0.765 ref=0.808 s_t=1.09
               var(pré-ativ) 1ª/meio/última: 2.6 1.3 0.0038 | ||grad W||: 0.25 0.12 1.9
fashion_L16   acc=0.765 base=0.095 ref=0.361 s_t=1.25
               var(pré-ativ) 1ª/meio/última: 2.5 1.2 0.0043 | ||grad W||: 0.47 0.12 1.4
fashion_L48   acc=0.207 base=0.095 ref=0.100 s_t=1.25
               var(pré-ativ) 1ª/meio/última: 2.7 1.2 0.0044 | ||grad W||: 8.9 0.46 0.92
cifar10_L4    acc=0.349 base=0.340 ref=0.229 s_t=1.25
               var(pré-ativ) 1ª/meio/última: 2.7 1.3 0.0042 | ||grad W||: 0.3 0.068 1.1
cifar10_L16   acc=0.183 base=0.100 ref=0.150 s_t=1.25
               var(pré-ativ) 1ª/meio/última: 2.5 1.2 0.0039 | ||grad W||: 0.92 0.1 0.95
cifar10_L48   acc=0.136 base=0.100 ref=0.090 s_t=1.25
               var(pré-ativ) 1ª/meio/última: 2.8 1.2 0.0043 | ||grad W||: 18 0.47 0.86
-----
ESCORE S = 117.0 (média=1.21, mínimo=1.07)
```



OBRIGADO!!

JONATHAN MOREIRA BARREIROS - 12421BCC034
KAIKE DE MORAIS CARVALHO - 12421BCC051